Library Management using Stack, Queue, Linked Lists and Binary Tree

Data Structures with Python (25CS33I) | Test IV | 50 Marks

AIM

To write a menu-driven Python program, using user-defined classes, to implement Stack, Queue, Singly Linked List, Doubly Linked List and Binary Tree for library management, with error handling.

PROCEDURE

1. **Stack (LIFO):** class Stack with a list. *push* adds at top, *pop* removes top, *peek* shows top, *is_empty* checks empty, *display* prints top to bottom. Empty stack gives an error message.
2. **Queue (FIFO):** class Queue with a list. *enqueue* adds at rear, *dequeue* removes from front, *display* prints waiting students. Empty queue gives "No student is waiting".
3. **Singly Linked List:** class Node (bid, title, next) and class SLL (head, size). *insert(pos)* and *delete(pos)* work on position: position 1 = beginning, last = end, other = middle. Invalid position gives an error. *traverse* prints all nodes; *search* returns the position of a Book ID or -1.
4. **Doubly Linked List:** class DNode (data, prev, next) and class DLL (head, tail, size). Insert and delete by position (update prev, next, head and tail pointers). *show(forward)* prints head to tail using next, or tail to head using prev.
5. **Binary Tree:** class TNode (data, left, right). Insert 50, 30, 70, 20, 40, 60, 80 (smaller goes left, larger goes right). *search* goes left or right from root. *delete*: leaf is removed, one child is replaced by its child, two children use the in-order successor.
6. **Traversals:** Recursive: In-order (L, Root, R), Pre-order (Root, L, R), Post-order (L, R, Root). Iterative: same orders using the Stack class (post-order with two stacks). Both outputs are compared and printed.
7. **Error handling:** *num()* repeats until a valid integer is entered (try/except). Empty stack/queue/list and invalid position raise exceptions, which every menu catches with try/except and prints the message. Result of every operation is displayed.

PROGRAM

```
# Library Management - Stack, Queue, Linked Lists, Binary Tree
def num(msg):                                   # valid integer input
    while True:
        try: return int(input(msg))
        except ValueError: print("Error: Invalid input!")

class Stack:                                    # LIFO
    def __init__(self): self.a = []
    def is_empty(self): return len(self.a) == 0
    def push(self, x): self.a.append(x)
    def pop(self):
        if self.is_empty(): raise IndexError("Stack is empty!")
        return self.a.pop()
    def peek(self):
        if self.is_empty(): raise IndexError("Stack is empty!")
        return self.a[-1]
    def display(self): print(self.a[::-1] if self.a else "Stack is empty!")

class Queue:                                    # FIFO
    def __init__(self): self.a = []
    def is_empty(self): return len(self.a) == 0
    def enqueue(self, x): self.a.append(x)
    def dequeue(self):
        if self.is_empty(): raise IndexError("No student is waiting!")
        return self.a.pop(0)
    def display(self): print(self.a if self.a else "No student is waiting!")

class Node:                                     # singly linked list node
    def __init__(self, bid, title): self.bid, self.title, self.next = bid, title, None

class SLL:
    def __init__(self): self.head, self.size = None, 0
    def insert(self, pos, bid, title):          # pos 1=begin, size+1=end, else middle
        if not 1 <= pos <= self.size + 1: raise ValueError("Invalid position!")
        n = Node(bid, title)
        if pos == 1: n.next, self.head = self.head, n
        else:
            t = self.head
            for _ in range(pos - 2): t = t.next
            n.next, t.next = t.next, n
        self.size += 1
    def delete(self, pos):
        if self.size == 0: raise IndexError("List is empty!")
        if not 1 <= pos <= self.size: raise ValueError("Invalid position!")
        if pos == 1: n = self.head; self.head = n.next
        else:
            t = self.head
            for _ in range(pos - 2): t = t.next
            n = t.next; t.next = n.next
        self.size -= 1
        return n
    def traverse(self):
        t = self.head
        if t is None: print("List is empty!")
        while t: print(t.bid, t.title); t = t.next
    def search(self, bid):                      # returns position or -1
        t, p = self.head, 1
        while t:
            if t.bid == bid: return p
            t, p = t.next, p + 1
        return -1

class DNode:                                    # doubly linked list node
    def __init__(self, data): self.data, self.prev, self.next = data, None, None

class DLL:
    def __init__(self): self.head, self.tail, self.size = None, None, 0
    def insert(self, pos, data):
        if not 1 <= pos <= self.size + 1: raise ValueError("Invalid position!")
        n, nxt = DNode(data), self.head
        for _ in range(pos - 1): nxt = nxt.next        # node now at pos (None if end)
        prv = nxt.prev if nxt else self.tail
        n.prev, n.next = prv, nxt
        if prv: prv.next = n
        else: self.head = n
        if nxt: nxt.prev = n
        else: self.tail = n
        self.size += 1
```

```
    def delete(self, pos):
        if self.size == 0: raise IndexError("List is empty!")
        if not 1 <= pos <= self.size: raise ValueError("Invalid position!")
        n = self.head
        for _ in range(pos - 1): n = n.next
        if n.prev: n.prev.next = n.next
        else: self.head = n.next
        if n.next: n.next.prev = n.prev
        else: self.tail = n.prev
        self.size -= 1
        return n.data
    def show(self, forward):
        r, t = [], (self.head if forward else self.tail)
        while t: r.append(t.data); t = t.next if forward else t.prev
        print(r if r else "List is empty!")

class TNode:                                # tree node
    def __init__(self, data): self.data, self.left, self.right = data, None, None

class Tree:
    def __init__(self): self.root = None
    def insert(self, n, x):
        if n is None: return TNode(x)
        if x < n.data: n.left = self.insert(n.left, x)
        elif x > n.data: n.right = self.insert(n.right, x)
        return n
    def search(self, x):
        t = self.root
        while t and t.data != x: t = t.left if x < t.data else t.right
        return t is not None
    def delete(self, n, x):
        if n is None: raise ValueError("Book ID not found!")
        if x < n.data: n.left = self.delete(n.left, x)
        elif x > n.data: n.right = self.delete(n.right, x)
        elif n.left is None: return n.right
        elif n.right is None: return n.left
        else:                               # two children: use in-order successor
            s = n.right
            while s.left: s = s.left
            n.data = s.data
            n.right = self.delete(n.right, s.data)
        return n
    def inorder(self, n): return self.inorder(n.left) + [n.data] + self.inorder(n.right) if n else []
    def preorder(self, n): return [n.data] + self.preorder(n.left) + self.preorder(n.right) if n else []
    def postorder(self, n): return self.postorder(n.left) + self.postorder(n.right) + [n.data] if n else []
    def inorder_it(self):
        r, s, c = [], Stack(), self.root
        while c or not s.is_empty():
            while c: s.push(c); c = c.left
            c = s.pop(); r.append(c.data); c = c.right
        return r
    def preorder_it(self):
        r, s = [], Stack()
        if self.root: s.push(self.root)
        while not s.is_empty():
            n = s.pop(); r.append(n.data)
            if n.right: s.push(n.right)
            if n.left: s.push(n.left)
        return r
    def postorder_it(self):                 # two stacks
        r, s1, s2 = [], Stack(), Stack()
        if self.root: s1.push(self.root)
        while not s1.is_empty():
            n = s1.pop(); s2.push(n)
            if n.left: s1.push(n.left)
            if n.right: s1.push(n.right)
        while not s2.is_empty(): r.append(s2.pop().data)
        return r

def stack_menu(s):
    while True:
        ch = num("\n[STACK] 1.Push 2.Pop 3.Peek 4.Display 5.Back: ")
        try:
            if ch == 1: s.push(input("Book: ")); print("Pushed.")
            elif ch == 2: print("Removed:", s.pop())
            elif ch == 3: print("Top:", s.peek())
            elif ch == 4: s.display()
            elif ch == 5: return
            else: print("Error: Invalid choice!")
        except IndexError as e: print("Error:", e)
def queue_menu(q):
```

```
    while True:
        ch = num("\n[QUEUE] 1.Enqueue 2.Dequeue 3.Display 4.Back: ")
        try:
            if ch == 1: q.enqueue(input("Student: ")); print("Added.")
            elif ch == 2: print("Served:", q.dequeue())
            elif ch == 3: q.display()
            elif ch == 4: return
            else: print("Error: Invalid choice!")
        except IndexError as e: print("Error:", e)

def sll_menu(l):
    while True:
        ch = num("\n[CATALOGUE] 1.InsBegin 2.InsEnd 3.InsMid 4.DelBegin "
                 "5.DelEnd 6.DelMid 7.Traverse 8.Search 9.Back: ")
        try:
            if ch in (1, 2, 3):
                bid, title = num("Book ID: "), input("Title: ")
                l.insert(1 if ch == 1 else l.size + 1 if ch == 2 else num("Position: "), bid, title)
                print("Inserted.")
            elif ch in (4, 5, 6):
                n = l.delete(1 if ch == 4 else l.size if ch == 5 else num("Position: "))
                print("Deleted:", n.bid, n.title)
            elif ch == 7: l.traverse()
            elif ch == 8:
                p = l.search(num("Book ID: "))
                print("Found at position", p) if p != -1 else print("Error: Book ID does not exist!")
            elif ch == 9: return
            else: print("Error: Invalid choice!")
        except (IndexError, ValueError) as e: print("Error:", e)

def dll_menu(l):
    while True:
        ch = num("\n[HISTORY] 1.InsBegin 2.InsEnd 3.InsMid 4.DelBegin "
                 "5.DelEnd 6.DelMid 7.Forward 8.Backward 9.Back: ")
        try:
            if ch in (1, 2, 3):
                d = input("Book: ")
                l.insert(1 if ch == 1 else l.size + 1 if ch == 2 else num("Position: "), d)
                print("Inserted.")
            elif ch in (4, 5, 6):
                print("Deleted:", l.delete(1 if ch == 4 else l.size if ch == 5 else num("Position: ")))
            elif ch in (7, 8): l.show(ch == 7)
            elif ch == 9: return
            else: print("Error: Invalid choice!")
        except (IndexError, ValueError) as e: print("Error:", e)

def tree_menu(t):
    while True:
        ch = num("\n[TREE] 1.Search 2.Delete 3.Recursive 4.Iterative+Compare 5.Back: ")
        try:
            if ch == 1: print("Found." if t.search(num("Book ID: ")) else "Error: Book ID does not exist!")
            elif ch == 2: t.root = t.delete(t.root, num("Book ID: ")); print("Deleted.")
            elif ch == 3:
                print("In-order  :", t.inorder(t.root))
                print("Pre-order :", t.preorder(t.root))
                print("Post-order:", t.postorder(t.root))
            elif ch == 4:
                for nm, a, b in (("In-order", t.inorder_it(), t.inorder(t.root)),
                                 ("Pre-order", t.preorder_it(), t.preorder(t.root)),
                                 ("Post-order", t.postorder_it(), t.postorder(t.root))):
                    print(nm, a, "-> same as recursive" if a == b else "-> different")
            elif ch == 5: return
            else: print("Error: Invalid choice!")
        except ValueError as e: print("Error:", e)

stack, queue, sll, dll, tree = Stack(), Queue(), SLL(), DLL(), Tree()
for x in (50, 30, 70, 20, 40, 60, 80): tree.root = tree.insert(tree.root, x)
while True:
    print("\n==== LIBRARY ====\n1.Stack 2.Queue 3.Singly LL 4.Doubly LL 5.Tree 6.Exit")
    ch = num("Choice: ")
    if ch == 1: stack_menu(stack)
    elif ch == 2: queue_menu(queue)
    elif ch == 3: sll_menu(sll)
    elif ch == 4: dll_menu(dll)
    elif ch == 5: tree_menu(tree)
    elif ch == 6: print("Thank you!"); break
    else: print("Error: Invalid choice!")
```

OUTPUT

```
==== LIBRARY ====
1.Stack 2.Queue 3.Singly LL 4.Doubly LL 5.Tree 6.Exit
Choice: 1
[STACK] 1.Push 2.Pop 3.Peek 4.Display 5.Back: 1
Book: Python Basics
Pushed.
[STACK] 1.Push 2.Pop 3.Peek 4.Display 5.Back: 1
Book: Data Structures
Pushed.
[STACK] 1.Push 2.Pop 3.Peek 4.Display 5.Back: 3
Top: Data Structures
[STACK] 1.Push 2.Pop 3.Peek 4.Display 5.Back: 4
['Data Structures', 'Python Basics']
[STACK] 1.Push 2.Pop 3.Peek 4.Display 5.Back: 2
Removed: Data Structures
[STACK] 1.Push 2.Pop 3.Peek 4.Display 5.Back: 2
Removed: Python Basics
[STACK] 1.Push 2.Pop 3.Peek 4.Display 5.Back: 2
Error: Stack is empty!
[STACK] 1.Push 2.Pop 3.Peek 4.Display 5.Back: 5
==== LIBRARY ====
1.Stack 2.Queue 3.Singly LL 4.Doubly LL 5.Tree 6.Exit
Choice: 2
[QUEUE] 1.Enqueue 2.Dequeue 3.Display 4.Back: 2
Error: No student is waiting!
[QUEUE] 1.Enqueue 2.Dequeue 3.Display 4.Back: 1
Student: Ravi
Added.
[QUEUE] 1.Enqueue 2.Dequeue 3.Display 4.Back: 1
Student: Amit
Added.
[QUEUE] 1.Enqueue 2.Dequeue 3.Display 4.Back: 2
Served: Ravi
[QUEUE] 1.Enqueue 2.Dequeue 3.Display 4.Back: 3
['Amit']
[QUEUE] 1.Enqueue 2.Dequeue 3.Display 4.Back: 4
==== LIBRARY ====
1.Stack 2.Queue 3.Singly LL 4.Doubly LL 5.Tree 6.Exit
Choice: 3
[CATALOGUE] 1.InsBegin 2.InsEnd 3.InsMid 4.DelBegin 5.DelEnd 6.DelMid 7.Traverse 8.Search 9.Back: 1
Book ID: abc
Error: Invalid input!
Book ID: 101
Title: DS
Inserted.
[CATALOGUE] 1.InsBegin 2.InsEnd 3.InsMid 4.DelBegin 5.DelEnd 6.DelMid 7.Traverse 8.Search 9.Back: 2
Book ID: 103
Title: OS
Inserted.
[CATALOGUE] 1.InsBegin 2.InsEnd 3.InsMid 4.DelBegin 5.DelEnd 6.DelMid 7.Traverse 8.Search 9.Back: 3
Book ID: 102
Title: Algo
Position: 2
Inserted.
[CATALOGUE] 1.InsBegin 2.InsEnd 3.InsMid 4.DelBegin 5.DelEnd 6.DelMid 7.Traverse 8.Search 9.Back: 7
101 DS
102 Algo
103 OS
[CATALOGUE] 1.InsBegin 2.InsEnd 3.InsMid 4.DelBegin 5.DelEnd 6.DelMid 7.Traverse 8.Search 9.Back: 8
Book ID: 102
Found at position 2
[CATALOGUE] 1.InsBegin 2.InsEnd 3.InsMid 4.DelBegin 5.DelEnd 6.DelMid 7.Traverse 8.Search 9.Back: 8
Book ID: 999
Error: Book ID does not exist!
[CATALOGUE] 1.InsBegin 2.InsEnd 3.InsMid 4.DelBegin 5.DelEnd 6.DelMid 7.Traverse 8.Search 9.Back: 6
Position: 9
Error: Invalid position!
[CATALOGUE] 1.InsBegin 2.InsEnd 3.InsMid 4.DelBegin 5.DelEnd 6.DelMid 7.Traverse 8.Search 9.Back: 6
Position: 2
Deleted: 102 Algo
[CATALOGUE] 1.InsBegin 2.InsEnd 3.InsMid 4.DelBegin 5.DelEnd 6.DelMid 7.Traverse 8.Search 9.Back: 4
Deleted: 101 DS
[CATALOGUE] 1.InsBegin 2.InsEnd 3.InsMid 4.DelBegin 5.DelEnd 6.DelMid 7.Traverse 8.Search 9.Back: 5
Deleted: 103 OS
[CATALOGUE] 1.InsBegin 2.InsEnd 3.InsMid 4.DelBegin 5.DelEnd 6.DelMid 7.Traverse 8.Search 9.Back: 9
==== LIBRARY ====
1.Stack 2.Queue 3.Singly LL 4.Doubly LL 5.Tree 6.Exit
Choice: 4
[HISTORY] 1.InsBegin 2.InsEnd 3.InsMid 4.DelBegin 5.DelEnd 6.DelMid 7.Forward 8.Backward 9.Back: 1
Book: Python
Inserted.
[HISTORY] 1.InsBegin 2.InsEnd 3.InsMid 4.DelBegin 5.DelEnd 6.DelMid 7.Forward 8.Backward 9.Back: 2
```

```
Book: Java
Inserted.
[HISTORY] 1.InsBegin 2.InsEnd 3.InsMid 4.DelBegin 5.DelEnd 6.DelMid 7.Forward 8.Backward 9.Back: 3
Book: C++
Position: 2
Inserted.
[HISTORY] 1.InsBegin 2.InsEnd 3.InsMid 4.DelBegin 5.DelEnd 6.DelMid 7.Forward 8.Backward 9.Back: 7
['Python', 'C++', 'Java']
[HISTORY] 1.InsBegin 2.InsEnd 3.InsMid 4.DelBegin 5.DelEnd 6.DelMid 7.Forward 8.Backward 9.Back: 8
['Java', 'C++', 'Python']
[HISTORY] 1.InsBegin 2.InsEnd 3.InsMid 4.DelBegin 5.DelEnd 6.DelMid 7.Forward 8.Backward 9.Back: 6
Position: 9
Error: Invalid position!
[HISTORY] 1.InsBegin 2.InsEnd 3.InsMid 4.DelBegin 5.DelEnd 6.DelMid 7.Forward 8.Backward 9.Back: 6
Position: 2
Deleted: C++
[HISTORY] 1.InsBegin 2.InsEnd 3.InsMid 4.DelBegin 5.DelEnd 6.DelMid 7.Forward 8.Backward 9.Back: 4
Deleted: Python
[HISTORY] 1.InsBegin 2.InsEnd 3.InsMid 4.DelBegin 5.DelEnd 6.DelMid 7.Forward 8.Backward 9.Back: 5
Deleted: Java
[HISTORY] 1.InsBegin 2.InsEnd 3.InsMid 4.DelBegin 5.DelEnd 6.DelMid 7.Forward 8.Backward 9.Back: 9
==== LIBRARY ====
1.Stack 2.Queue 3.Singly LL 4.Doubly LL 5.Tree 6.Exit
Choice: 5
[TREE] 1.Search 2.Delete 3.Recursive 4.Iterative+Compare 5.Back: 3
In-order  : [20, 30, 40, 50, 60, 70, 80]
Pre-order : [50, 30, 20, 40, 70, 60, 80]
Post-order: [20, 40, 30, 60, 80, 70, 50]
[TREE] 1.Search 2.Delete 3.Recursive 4.Iterative+Compare 5.Back: 1
Book ID: 60
Found.
[TREE] 1.Search 2.Delete 3.Recursive 4.Iterative+Compare 5.Back: 1
Book ID: 99
Error: Book ID does not exist!
[TREE] 1.Search 2.Delete 3.Recursive 4.Iterative+Compare 5.Back: 2
Book ID: 99
Error: Book ID not found!
[TREE] 1.Search 2.Delete 3.Recursive 4.Iterative+Compare 5.Back: 2
Book ID: 50
Deleted.
[TREE] 1.Search 2.Delete 3.Recursive 4.Iterative+Compare 5.Back: 3
In-order  : [20, 30, 40, 60, 70, 80]
Pre-order : [60, 30, 20, 40, 70, 80]
Post-order: [20, 40, 30, 80, 70, 60]
[TREE] 1.Search 2.Delete 3.Recursive 4.Iterative+Compare 5.Back: 4
In-order [20, 30, 40, 60, 70, 80] -> same as recursive
Pre-order [60, 30, 20, 40, 70, 80] -> same as recursive
Post-order [20, 40, 30, 80, 70, 60] -> same as recursive
[TREE] 1.Search 2.Delete 3.Recursive 4.Iterative+Compare 5.Back: 5
==== LIBRARY ====
1.Stack 2.Queue 3.Singly LL 4.Doubly LL 5.Tree 6.Exit
Choice: 9
Error: Invalid choice!
==== LIBRARY ====
1.Stack 2.Queue 3.Singly LL 4.Doubly LL 5.Tree 6.Exit
Choice: 6
Thank you!
```

Result: The menu-driven Library Management program was written and executed successfully.